5 İndeks məlumatı tapmağı necə dəyişir

Kafedə on minlərlə sifariş var. “Müştəri 7-nin sifarişlərini gətir” deyirik. Bazada uyğun axtarış quruluşu yoxdursa çoxlu sifariş səhifəsinə baxmaq lazım ola bilər.

Kitabda bir mövzunu axtarmağı düşün. Bütün səhifələri bir-bir oxumaq olar. Yaxud arxadakı göstəricidən mövzunun yerini tapıb həmin səhifəyə keçmək olar.

Baza **indeksi** həmin göstəriciyə bənzəyən ayrıca axtarış quruluşudur. Axtarılan dəyərlə məlumatın yerini əlaqələndirir.

İndeksin verdiyi fikir

```
Indeks yoxdur:
    Coxlu melumat sehifesine bax, uygun setirleri sec.

Uygun indeks var:
    Axtaris qurulusunda lazim olan hisseye cat.
    Uygun melumati oxu.
```

İndeksin məqsədi məlumatın özünü dəyişmək deyil, **uyğun sorğuda görülən oxuma işini azaltmaqdır**.

Amma kitabın yarısını oxumaq istəyirsənsə hər səhifəyə ayrıca göstəricidən getmək sərfəli olmaya bilər. Bazada da çoxlu nəticə qaytaran sorğu üçün başqa yol daha ucuz ola bilər.

Sütuna indeks verəndə nə yaranır

Bu SQL Server cədvəlini düşün:

Id	CustomerId	Total
42	7	30
43	7	10
44	9	20

T-SQL

```
CREATE INDEX IX_Orders_Customer
ON dbo.Orders(CustomerId);
```

Bu əmr `CustomerId` üzrə indeks yaradır. Sütunun içindəki 7 və 9 dəyərləri dəyişmir. Əlavə quruluş təxminən “müştəri 7 üçün bu sətirlərə, 9 üçün o sətrə bax” imkanı verir.

Sadələşdirilmiş xəritə

```
CustomerId 7 -> Order 42, Order 43
CustomerId 9 -> Order 44
```

Bu xəritə axtarış fikrini göstərir; real daxili quruluş bir qədər sonra izah olunur.

İndi `WHERE CustomerId = 7` şərti ilə sorğu gələndə baza bu indeksdən istifadə etməyi qiymətləndirə bilər. `WHERE` sətirləri seçən şərtdir. İndeksin mövcudluğu mütləq həmin indeksin seçilməsi demək deyil.

Yadda saxla: sütuna indeks vermək, o sütuna əlavə axtarış quruluşu yaratmaqdır. Hər sorğuya sürət zəmanəti vermək deyil.

B plus tree böyük siyahını hissələrə bölür

Adi SQL Server rowstore indeksləri **B+ tree** adlı balanslı ağac quruluşundan istifadə edir. Rowstore məlumatın sətir əsaslı saxlanmasıdır.

Ağacda yuxarıdan aşağı hər dəfə uyğun aralığı seçirik. Hər sətri əvvəldən yoxlamaq əvəzinə uyğun hissəyə yaxınlaşırıq:

Sadələşdirilmiş quruluş

```
Baslangic: [1..100] [101..200] [201..300]
                       |
                       v
                [101..150] [151..200]
                                |
                                v
                       [171, 172, 173, 174, ...]
```

Başlanğıc hissə **root**, aralıq hissə **branch**, son səviyyə **leaf** adlanır. Hər real qovşaqda çoxlu açar ola bilər. Bu, hər qovşağı yalnız iki hissəyə bölünən sadə binary ağac deyil.

Page, səhifə, bazanın məlumat saxladığı blokdur. SQL Server-in adi rowstore səhifəsi 8 KB-dır və içində bir neçə sətir ola bilər. Buna görə bir milyon sətirə baxmaq mütləq bir milyon disk oxunuşu demək deyil.

O(log n) böyümə barədə nəzəri ifadədir: məlumat artdıqca ağacda yolu tapmaq işi sətirlərin hamısını gəzmək qədər artmır. Bu ifadə dəqiq saniyə və disk oxunuş sayı vermir. Tapıldıqdan sonra nəticələrin özünü oxumağın da xərci vardır.

Clustered və nonclustered fərqi

İndi iki quruluşu ayıraq. Bu səhifə SQL Server-in adi sətir əsaslı indekslərinə aiddir.

Clustered index: ağacın yarpaq səviyyəsində cədvəlin məlumat sətirləri yerləşir. Yəni son səviyyəyə çatanda məlumatın özünə çatırıq. Bu səbəbdən cədvəldə bir clustered rowstore index olur.

Nonclustered index: ayrıca ağacdır. Yarpaqda indeks məlumatı və əsas sətri tapmağa kömək edən yer göstəricisi saxlanır.

Nonclustered indeksdən istifadə nümunəsi

```
CustomerId 7-ni indeksde tap.
    |
    v
Uyqun sifarisin yerini elde et.
    |
    v
Lazim olan diger sutunlari esas setirden oxu.
```

Yer göstəricisinə **locator** deyilir. Cədvəldə clustered index varsa nonclustered indeks clustered açardan istifadə edir. Clustered index olmayan SQL Server cədvəli **heap** adlanır; onda locator sətrin yerini başqa qayda ilə göstərir.

İndeksdən sonra çatışmayan məlumat üçün əsas sətrə getməyə **lookup** deyilir. Az nəticədə ucuz, çox nəticədə baha ola bilər.

Clustered index açar sırasına görə məntiqi quruluş yaradır. Bütün səhifələrin diskdə daim yan-yana olması və SELECT nəticəsinin avtomatik sıralı gəlməsi zəmanəti deyil. Cavab sırası üçün `ORDER BY` yazılır.

Primary key və unique indeks eyni sualı cavablandırmır

Primary key, əsas açar, cədvəldə hər sətri fərqləndirən qaydadır. Onun dəyəri təkrarlanmamalı və NULL olmamalıdır. NULL burada dəyərin olmadığını və ya naməlum olduğunu bildirən SQL işarəsidir.

Unique index müəyyən açarın təkrarlanmasına icazə vermir. Məsələn, sistemin qaydası email-in təkrarlanmamasıdırsa:

T-SQL

```
CREATE UNIQUE INDEX UX_Customers_Email
ON dbo.Customers(Email);
```

İki paralel əməliyyat eyni email-i yazmağa çalışanda unikal qaydanı baza özü qoruyur. Tətbiqdə “əvvəl yoxla, sonra yaz” demək təkbaşına bu zəmanəti vermir: iki əməliyyat da eyni anda “yoxdur” cavabı ala bilər.

Primary key məntiqi məlumat qaydasıdır. Clustered isə məlumatın saxlanma quruluşudur. SQL Server primary key üçün dəstək indeksi yaradır, amma primary key-in mütləq clustered olması tələb deyil.

Foreign key, xarici açar, başqa cədvəldəki uyğun açara istinad qaydasıdır. SQL Server-də belə qaydanın mövcudluğu həmin sütun üçün mütləq avtomatik əlavə indeks yaradıldığı demək deyil.

Nəticə: “nəyi unikal saxlamalıyam?”, “hansı əlaqə etibarlıdır?” və “sorğunu necə sürətləndirim?” əlaqəli, amma ayrı suallardır.

İki sütunlu indeksdə sıra niyə dəyişir

Telefon kitabçasında əvvəl soyad, sonra ad sırası olduğunu düşün. Soyadı biləndə uyğun hissəyə rahat gedirsən. Tək adı biləndə eyni ad müxtəlif soyadların arasında yayılır.

Composite index bir neçə sütunun birlikdə açar olduğu indeksdir. `(CustomerId, CreatedAt)` əvvəl müştəriyə, həmin müştərinin içində tarixə görə qurulur.

İndeksin məntiqi sırası

```
Musteri 7
    2026-10-20
    2026-10-21
Musteri 9
    2026-10-19
    2026-10-23
```

Müştəri 7-nin müəyyən tarixdən sonrakı sifarişləri bu quruluşa uyğundur. Yalnız tarixlə axtarışda isə həmin tarixlər müxtəlif müştəri hissələrində yayılır.

T-SQL

```
CREATE INDEX IX_Orders_Customer_Date
ON dbo.Orders(CustomerId, CreatedAt);
```

“Ən çox fərqli dəyəri olan sütun həmişə əvvəl olmalıdır” qaydasını əzbərləmə. Əvvəl hansı sorğunun hansı bərabərlik və aralıq şərtlərindən istifadə etdiyini müəyyən et. Sonra sıralamanı və başqa sorğuları nəzərə al.

Yadda saxla: sütunların siyahısı qədər həmin siyahının sırası da indeksin hansı sorğuya rahat yol verdiyini dəyişir.

INCLUDE əsas sətrə əlavə gedişi azalda bilər

Sorğu müştəri və tarixlə axtarır, amma cavabda Total məbləğini də istəyir. İndeksdə Total yoxdursa baza onu əsas sətirdən ayrıca oxuya bilər.

T-SQL

```
CREATE INDEX IX_Orders_Customer_Date
ON dbo.Orders(CustomerId, CreatedAt)
INCLUDE(Total);
```

`INCLUDE(Total)` məbləği də indeksin yarpaq səviyyəsində saxlayır. Amma Total axtarış açarlarının sırasına əlavə edilmir.

Bu sorğunun lazım olan məlumatı

```
Filtr:  CustomerId ve CreatedAt.
Cavab:  CreatedAt ve Total.

Hamisi bu indeksde tapila biler.
```

Bir sorğunun ehtiyac duyduğu bütün məlumat indeksdədirsə həmin sorğu üçün ona **covering index** deyilir. Covering “əhatə edən” deməkdir. Eyni indeks başqa, daha çox sütun istəyən sorğunu əhatə etməyə bilər.

Hər sütunu INCLUDE etmək də pulsuz deyil. İndeks böyüyür, daha çox yer tutur və həmin məlumat dəyişəndə əlavə iş yaradır.

Nəticə: açar sütunları axtarış və sıra üçün, INCLUDE sütunları isə nəticədə lazım olan əlavə məlumat üçün seçilir.

İndeks niyə bəzən seçilmir

Hansı yolu seçən hissəyə **optimizer** deyilir. O, sorğunu yerinə yetirmək üçün mümkün yolların xərcini qiymətləndirir.

Index seek uyğun açar aralığından başlamaqdır. **Index scan** indeksin böyük hissəsini və ya hamısını gəzməkdir. **Table scan** cədvəl məlumatını gəzməkdir.

Milyon sifarişin yalnız 3-ü lazımdırsa dar indeks axtarışı sərfəli ola bilər. Sifarişlərin 90 faizi lazımdırsa hər sətrə ayrıca lookup etməkdənsə geniş oxuma daha ucuz ola bilər. Seek həmişə yaxşı, scan həmişə pis deyil.

Selectivity şərtin məlumatın nə qədərini seçməsini ifadə edir. **Cardinality** kontekstdən asılı olaraq fərqli dəyər sayını və ya nəticə sətir sayını bildirə bilər.

Məsələn, iki statusdan yalnız biri cəmi 100 sətrə aiddirsə, həmin nadir hissə üçün indeks faydalı ola bilər. “Bool və az variantlı sütuna indeks həmişə mənasızdır” düzgün deyil.

Filtered index yalnız müəyyən şərtə uyğun sətirləri saxlayan SQL Server indeksidir. PostgreSQL-də oxşar anlayış partial index adlanır.

Optimizer-in dəyərlərin paylanmasını təxmin etməsinə kömək edən məlumata **statistics**, statistika deyilir. Səhv təxmin uyğun olmayan plan seçiminə səbəb ola bilər.

Oxuma qazancının yazma xərci var

İndeks cədvəllə uyğun qalmalıdır. Buna görə əlavə iş yaranır:

Dəyişiklik	İndeksdə görülən iş
INSERT	Yeni sətrə uyğun indeks məlumatı əlavə edilir.
İndeksdə saxlanan dəyərə UPDATE	Müvafiq indeks məlumatı yenilənir.
DELETE	Sətrin indeks məlumatı da silinmə baxımına düşür.

İndeks ayrıca disk sahəsi tutur. İstifadə olunan səhifələr yaddaşda yer tutur. Dəyişikliklərin bərpa üçün yazıldığı **transaction log** yükü də arta bilər.

Sütunun üzərində hesablama aparmaq axtarışı çətinləşdirə bilər:

T-SQL şərtləri

```
-- Ilk forma her tarixden il hesablayir:
WHERE YEAR(CreatedAt) = 2026

-- Ikinci forma tarix araligini birbasa gosterir:
WHERE CreatedAt >= '20260101'
  AND CreatedAt <  '20270101'
```

YEAR tarixdən ili çıxarır. İkinci forma uyğun indeks üçün daha rahat aralıq yarada bilər. İndeksdə səmərəli axtarış şərti kimi istifadə oluna bilən yazılışa **SARGable** deyilir. Bəzi hesablanmış sütun indeksləri kimi istisnalar da var; “funksiya varsa heç bir indeks işləməz” hökmü də doğru deyil.

PostgreSQL-də xüsusi indekslər nə üçündür

Bu səhifədəki növlər PostgreSQL üçündür. Onların adlarını SQL Server-in bütün cədvəllərinə eyni qayda ilə tətbiq etmək olmaz.

B-tree sıralana bilən dəyərlərdə bərabərlik və aralıq axtarışı üçün ümumi seçimdir. **Hash** dəyərdən hesablanan kodla sadə bərabərlik axtarışına xidmət edir; adi tarix aralığı sıralaması vermir.

GIN dəyərin içindəki hissədən onu saxlayan sətirlərə keçid yaradır. Məsələn, etiket siyahıları varsa:

GIN fikri

```
"cay" etiketi  -> 1, 4, 9 nomreli mehsullar.
"sirin" etiketi -> 2, 4 nomreli mehsullar.
```

GiST müxtəlif tiplərə uyğun axtarış qurmağa imkan verən çərçivədir. Həndəsi məlumatda bölgələrdən namizəd yerləri tapmaq nümunədir. **SP-GiST** məkanı hissələrə ayıran başqa quruluşları dəstəkləyir. Hansı müqayisələrin işləməsini konkret tip üçün operator qaydaları müəyyən edir.

BRIN hər sətir üçün ayrıca yer saxlamaq əvəzinə səhifə aralıqlarının xülasəsini saxlayır. Tarixlər fiziki sıra ilə yaxın gedirsə bəzi aralıqların sorğuya uyğun gəlmədiyini tez müəyyən edə bilər. Namizəd aralıqdakı real sətirlər yenidən yoxlanır.

Nəticə: indeks növünü məlumatın və axtarışın formasına görə seçirik. Bütün məlumat növlərinə bir indeks eyni dərəcədə uyğun deyil.

Düzgün indeks qərarını necə verək

Kafedə əsas sorğumuz belə olsun: müştərinin müəyyən tarixdən sonrakı sifarişləri, tarix və məbləğlə birlikdə göstərilsin.

T-SQL nümunəsi

```
SELECT CreatedAt, Total
FROM dbo.Orders
WHERE CustomerId = @CustomerId
  AND CreatedAt >= @From;
```

`@CustomerId` müştəri nömrəsi, `@From` başlanğıc tarix parametridir. `AND` hər iki şərtin doğru olmasını tələb edir. Bu sorğu üçün `(CustomerId, CreatedAt) INCLUDE(Total)` məntiqli namizəddir. Son qərar ölçmə ilə verilir.

SQL Server-də **execution plan** sorğunun hansı addımlarla icra olunduğunu göstərir. `SET STATISTICS IO ON` oxunan səhifələr barədə məlumat verir. **Logical read** səhifəyə yaddaş üzərindən müraciətdir. **Physical read** səhifəni diskdən gətirmək ehtiyacıdır; səhifə artıq yaddaşdadırsa yenidən disk oxunuşu lazım olmaya bilər.

PostgreSQL-də `EXPLAIN` planı, `EXPLAIN ANALYZE` isə sorğunu həqiqətən işlədərək icra məlumatını göstərir. Yazma sorğusunda ANALYZE də yazmanı icra edir.

Özünü yoxla

İndeks qoyandan sonra yalnız “seek göründü” demə. **Oxunan səhifə azaldımı, cavab vaxtı yaxşılaşdımı, əlavə yazma xərci qəbul edilirmi?** Bunları müqayisə et. Sabit “100 dəfə sürətlənəcək” vədi vermək olmaz.